CAMPUS TRANSPORT ALLOCATION

Python Programming Case Study

1. Case Study

This case study estimates the number of buses needed to serve passengers on multiple campus routes. Given the passenger count for each route and a fixed bus capacity, the program calculates the required buses and reports the number of passengers served and unserved.

2. Problem Statement

Campus transport coordinators need to assign enough buses to meet route demand. Calculating bus requirements manually can lead to insufficient capacity or inefficient planning. A program can compute the required number of buses for each route and summarize service coverage.

3. Objective

To calculate the minimum whole number of buses needed per route based on passenger demand and bus capacity, then report served and unserved passengers.

4. Assumptions

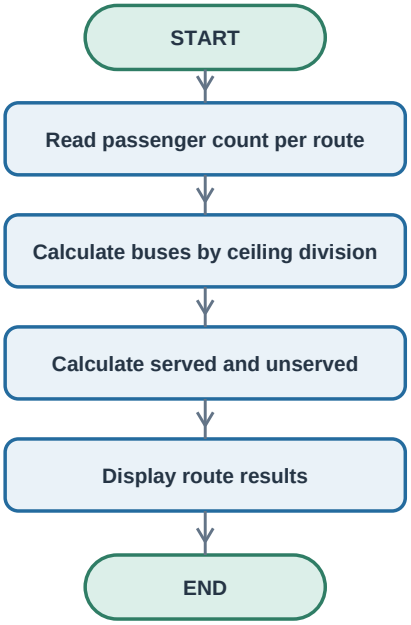
- Passenger counts are non-negative whole numbers.
- Every bus has the same fixed capacity of 40 passengers.
- The number of buses is rounded up so all passengers can be accommodated.
- All buses assigned to a route are available and can be filled up to capacity.
- The sample calculations assume sufficient buses are available; no fleet-wide bus limit is modeled.

5. Algorithm

1. Store passenger counts for each route and set bus capacity to 40.
2. For each route, read the passenger count.
3. Calculate buses needed using ceiling division: (passengers + capacity - 1) // capacity.
4. Calculate passengers served as the smaller of demand and total available seats.
5. Calculate unserved passengers as demand minus served passengers.
6. Display the route number, buses needed, and unserved passengers.

6. Flowchart

For each route, the program calculates the bus requirement and reports service coverage.



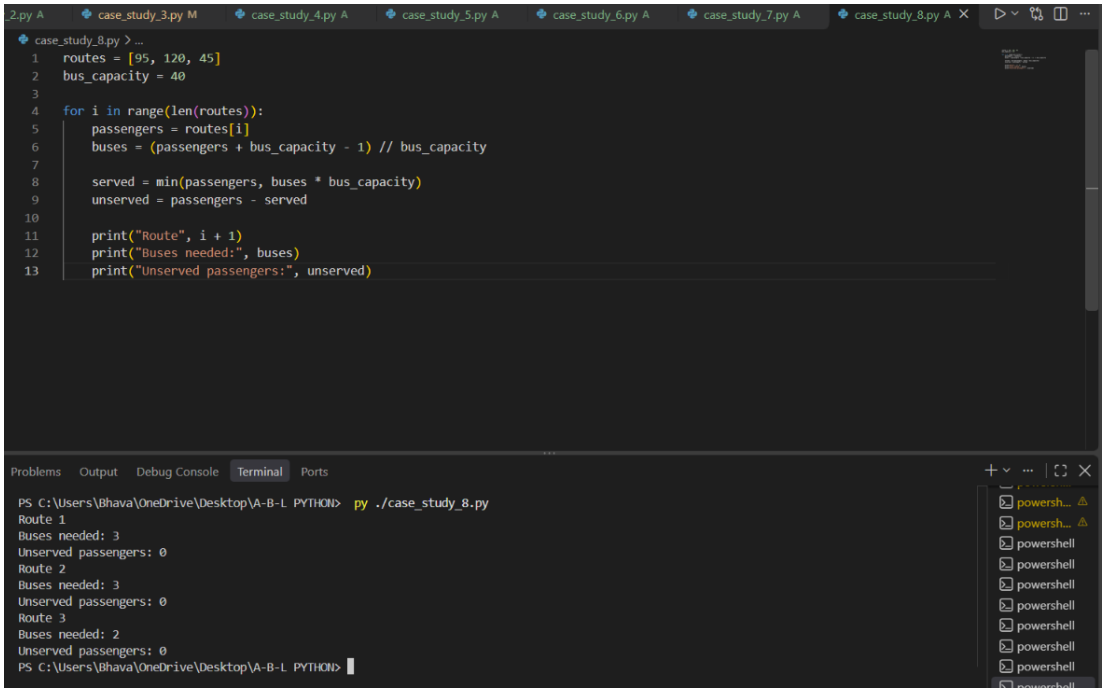
7. Pseudocode

```
BEGIN
  routes = passenger counts by route
  bus_capacity = 40

  FOR each passenger count in routes
    buses = (passengers + bus_capacity - 1) // bus_capacity
    served = MIN(passengers, buses * bus_capacity)
    unserved = passengers - served
    display route, buses, unserved
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows passenger counts of 95, 120, and 45 with a bus capacity of 40. The program reports 3 buses for Route 1, 3 for Route 2, and 2 for Route 3. In this example, the calculated bus capacity is enough to serve all passengers, so unserved passengers are zero on every route.



9. Conclusion

The program applies ceiling division to determine the number of buses required for each route. For the sample passenger counts, 8 buses are needed in total, and all 260 passengers can be served with the assumed capacity. The route-by-route calculation provides a simple planning aid; a larger system could also account for a limited fleet, departure schedules, and different vehicle capacities.

FOOD WASTE CALCULATION AND PERCENTAGE

Python Programming Case Study

1. Case Study

This case study uses Python to measure daily food waste. For each day, the program compares the quantity of food prepared with the quantity served, finds the leftover amount, and calculates waste as a percentage of the food prepared.

2. Problem Statement

Food service providers need a clear way to track how much prepared food is left unused. Without calculating the amount and percentage of waste each day, it is difficult to compare daily results and identify opportunities to reduce waste.

3. Objective

To calculate the quantity and percentage of food wasted each day from the prepared and served quantities, then display the results for each day.

4. Assumptions

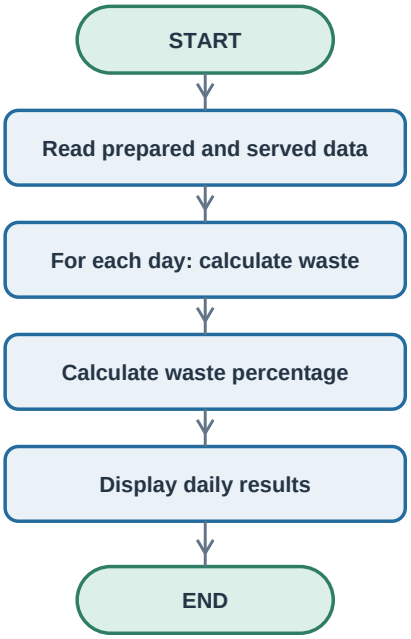
- The prepared and served lists contain the same number of daily entries.
- Each list position refers to the same day in both lists.
- Quantities are numeric and measured in the same unit.
- The amount served does not exceed the amount prepared, and the prepared amount is greater than zero.
- Waste percentage is calculated relative to the prepared amount.

5. Algorithm

1. Store the daily prepared quantities and served quantities in two lists.
2. Loop through each day by its list index.
3. Subtract the served quantity from the prepared quantity to find food waste.
4. Divide waste by the prepared quantity and multiply by 100 to find the waste percentage.
5. Display the day number, waste amount, and waste percentage.

6. Flowchart

The program processes each day in turn, computes both waste measures, and prints the results.



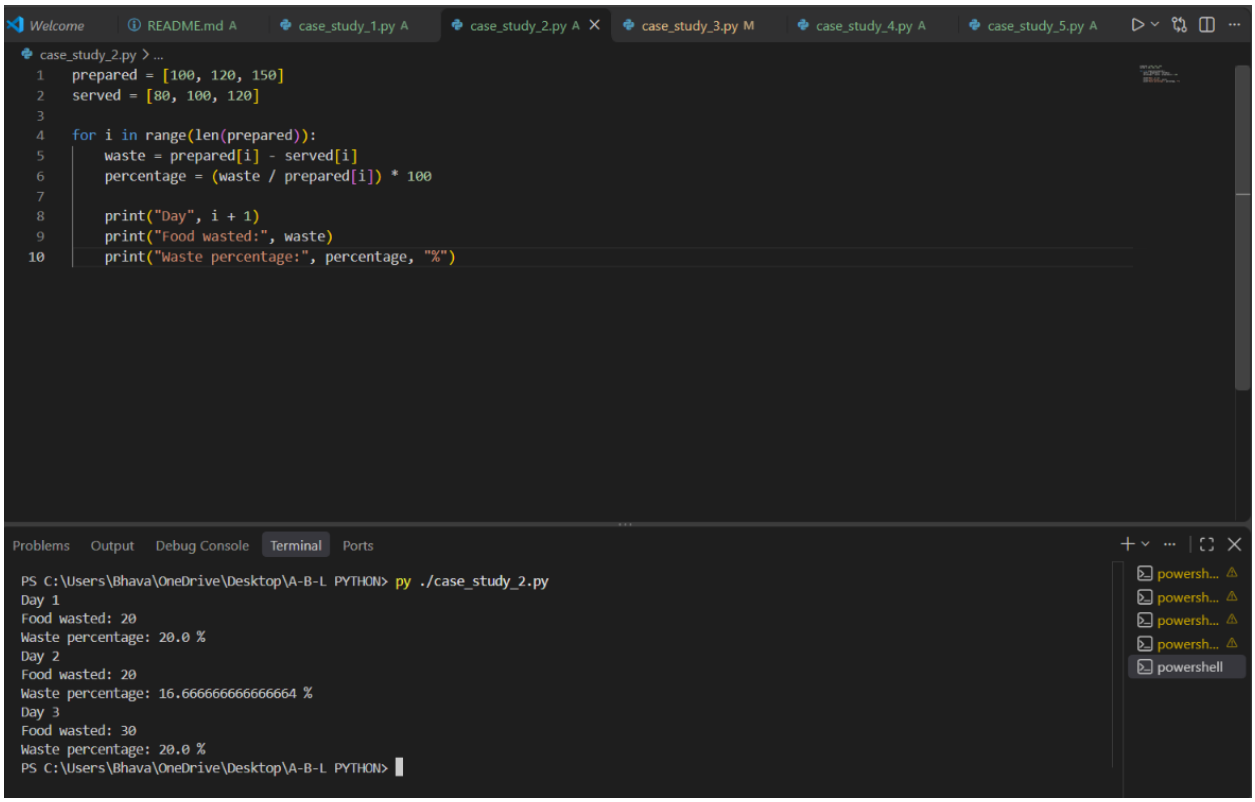
7. Pseudocode

```
BEGIN
  prepared = daily prepared quantities
  served = daily served quantities

  FOR i from 0 to length(prepared) - 1
    waste = prepared[i] - served[i]
    percentage = (waste / prepared[i]) * 100
    display day number, waste, percentage
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows the Python source code and terminal results. For the three days, the program reports 20 units (20%) wasted, 20 units (about 16.67%) wasted, and 30 units (20%) wasted.



9. Conclusion

The program calculates the daily leftover food and expresses it as a percentage of the amount prepared. In the sample, the most food wasted by quantity is 30 units on Day 3; the highest percentage is approximately 20%, shared by Days 1 and 3. These measurements give a simple basis for monitoring and reducing food waste.

HACKATHON TEAM FORMATION AND RANKING

Python Programming Case Study

1. Case Study

This case study presents a simple way to compare hackathon teams using the scores of their members. The program stores each team with a list of member scores, calculates a total for every team, and ranks the teams from the highest total to the lowest.

2. Problem Statement

When several teams receive individual member scores, comparing the teams manually can be time-consuming and may lead to ordering mistakes. A program is needed to calculate each team's combined score and display an accurate ranking.

3. Objective

To use Python dictionaries, lists, loops, the **sum()** function, and sorting to calculate team totals and produce a descending team ranking.

4. Assumptions

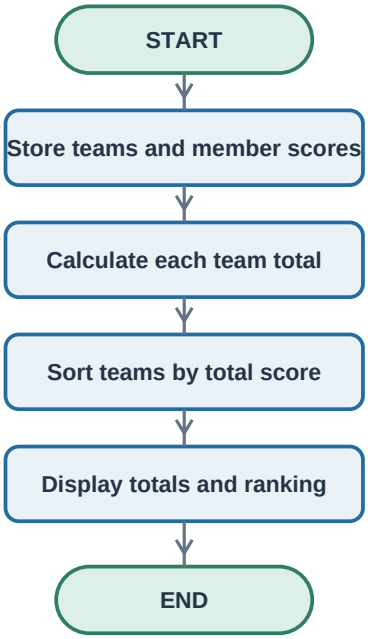
- Each team has a unique name.
- Each team has one or more numeric member scores.
- A team's total is the sum of its member scores.
- A higher total means a higher rank.
- If totals tie, Python's stable sort preserves the teams' original input order; the example has no tie.

5. Algorithm

1. Create a dictionary containing team names and their member scores.
2. Visit each team and calculate its total using **sum(scores)**.
3. Sort the dictionary entries by the sum of their scores in descending order.
4. Print the teams' totals.
5. Enumerate the sorted teams starting at rank 1 and print the ranking.

6. Flowchart

The process starts with the team data, calculates totals, sorts teams by score, and displays the results.



7. Pseudocode

```
BEGIN
  teams = team names mapped to lists of member scores

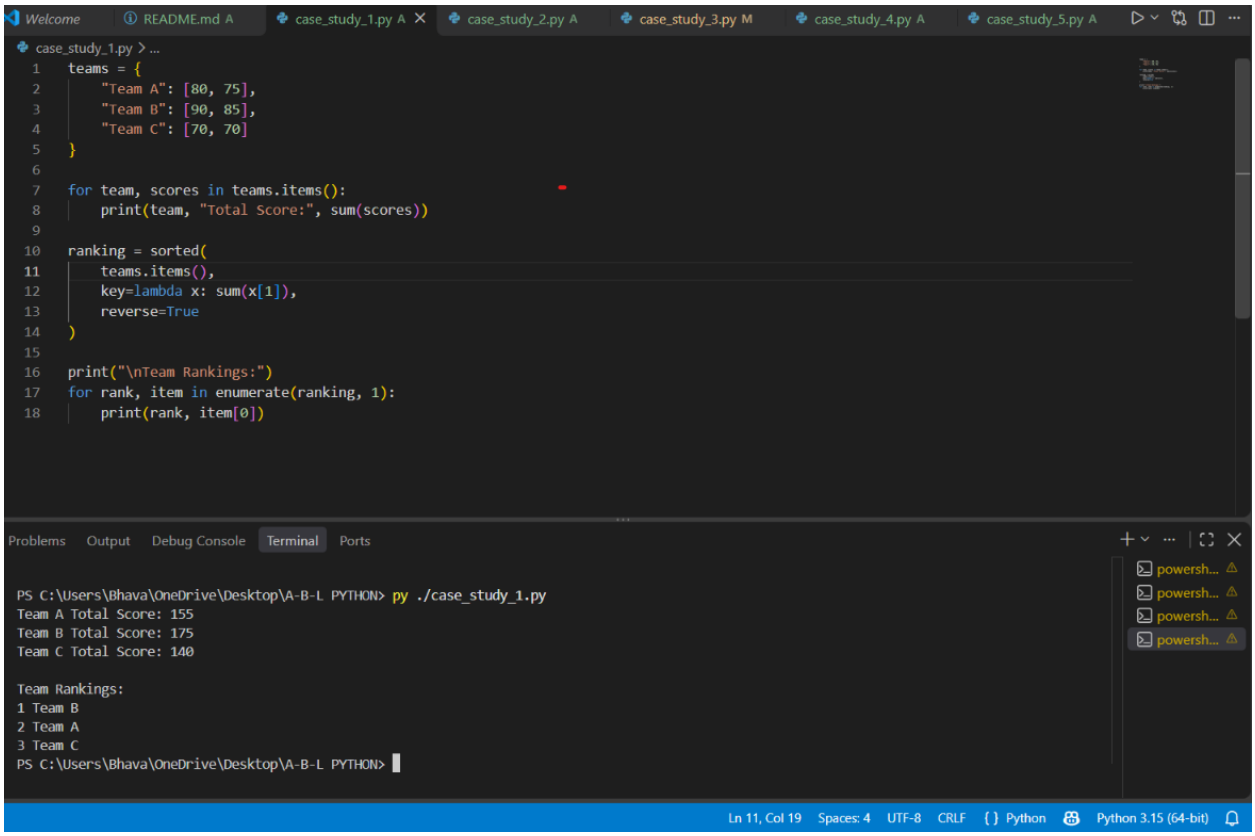
  FOR each team and its scores in teams
    total = sum(scores)
    display team and total
  END FOR

  ranking = teams sorted by sum(scores), highest first

  FOR each item in ranking, with rank starting at 1
    display rank and team name
  END FOR
END
```

8. Program and Output Screenshot

The screenshot below shows the Python source code and its terminal output. The totals are Team A = 155, Team B = 175, and Team C = 140; the resulting ranking is Team B, Team A, then Team C.



9. Conclusion

The program correctly adds the member scores for each team and sorts the teams from highest to lowest total. For the sample data, Team B ranks first with 175 points, Team A ranks second with 155 points, and Team C ranks third with 140 points. This approach can make basic hackathon team comparisons faster and more consistent.

LIBRARY BORROWING AND FINE RECONCILIATION

Python Programming Case Study

1. Case Study

This case study calculates late-return fines for library books. Each record pairs a book ID with the number of days it was returned late. The program applies a fixed fine rate per day, displays the fine for each book, and accumulates a total fine amount.

2. Problem Statement

When several borrowed books are returned late, library staff need to calculate each fine consistently and determine the total amount owed. Repeated manual calculations can introduce errors, so the process can be automated using a short program.

3. Objective

To process book IDs and days-late values, calculate each book's fine using a fixed daily rate, and report both individual fines and the combined total.

4. Assumptions

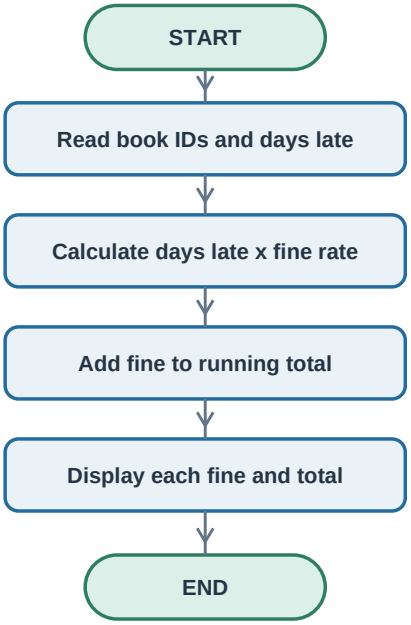
- Each book record contains a book ID and a non-negative number of days late.
- The daily fine rate is fixed at 5 units of currency per late day.
- Each book is assessed independently; no maximum fine or grace period is applied.
- The total fine is the sum of all individual book fines.
- The screenshot does not specify a currency symbol, so amounts are described as currency units.

5. Algorithm

1. Store the book IDs and their days-late values as pairs.
2. Set the fine rate per day to 5 and initialize the total fine to zero.
3. For each book, multiply days late by the daily rate.
4. Add the individual fine to the running total.
5. Display the book ID and its fine.
6. After all books are processed, display the total fine.

6. Flowchart

The program calculates a fine for each record, accumulates the amount, and displays the final total.



7. Pseudocode

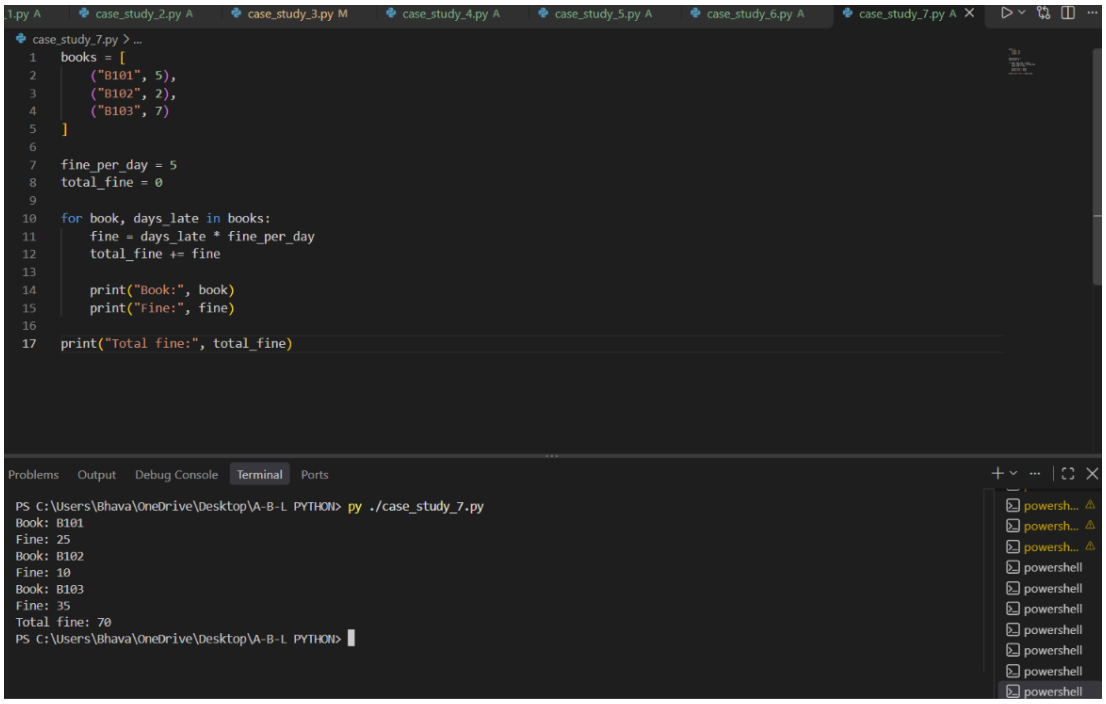
```
BEGIN
  books = list of (book ID, days late) pairs
  fine_per_day = 5
  total_fine = 0

  FOR each (book, days_late) in books
    fine = days_late * fine_per_day
    total_fine = total_fine + fine
    display book and fine
  END FOR

  display total_fine
END
```

8. Program and Output Screenshot

The screenshot shows three book records: B101 is 5 days late, B102 is 2 days late, and B103 is 7 days late. At 5 currency units per day, their fines are 25, 10, and 35, producing a total fine of 70.



9. Conclusion

The program applies the same daily fine rate to each late book and keeps an accurate running total. For the displayed records, the individual fines are 25, 10, and 35 currency units, and the total due is 70. This provides a simple, repeatable way to reconcile late-return fines.

MANUFACTURING QUALITY CONTROL MATRIX

Python Programming Case Study

1. Case Study

This case study reviews manufacturing batches using a matrix of production and defective-unit counts. For each batch, the program calculates the defect percentage and compares it with a set quality limit. Batches above the limit are flagged for attention.

2. Problem Statement

Manufacturers need to monitor product defects across production batches. Comparing raw defect counts alone can be misleading when batch sizes differ. A percentage-based measure and a consistent threshold help identify batches that may require investigation.

3. Objective

To calculate each batch’s defect percentage from its production and defect counts, display the percentage, and report a warning when the percentage exceeds 10%.

4. Assumptions

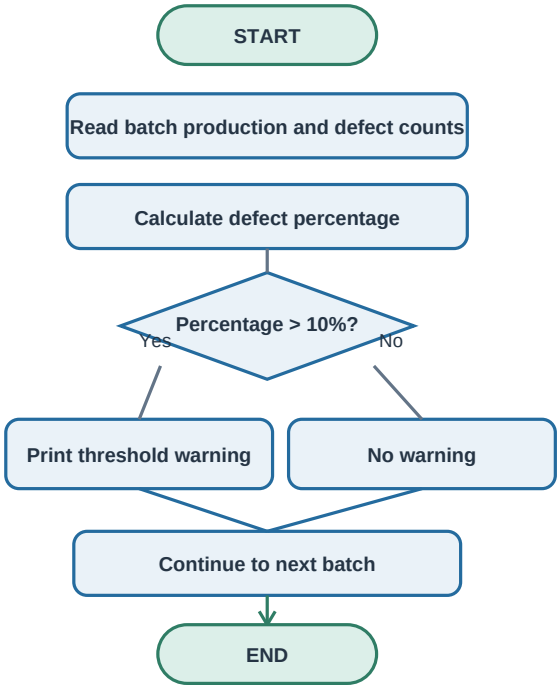
• Each matrix row contains [total units produced, defective units].
• Total production is greater than zero, and defective units are non-negative.
• Defective units do not exceed total production.
• The quality threshold is fixed at 10%.
• A warning is printed only when defect percentage is greater than 10%; a value exactly equal to 10% would not trigger the warning.

5. Algorithm

1. Store each batch as a pair of production and defective-unit counts.
2. Set the quality limit to 10%.
3. For each batch, read total production and defective units.
4. Calculate (defective units / total units) × 100.
5. Display the batch number and defect percentage.
6. If the percentage is above the limit, print a threshold warning.

6. Flowchart

Each batch is assessed against the 10% defect threshold; batches over the limit receive a warning.



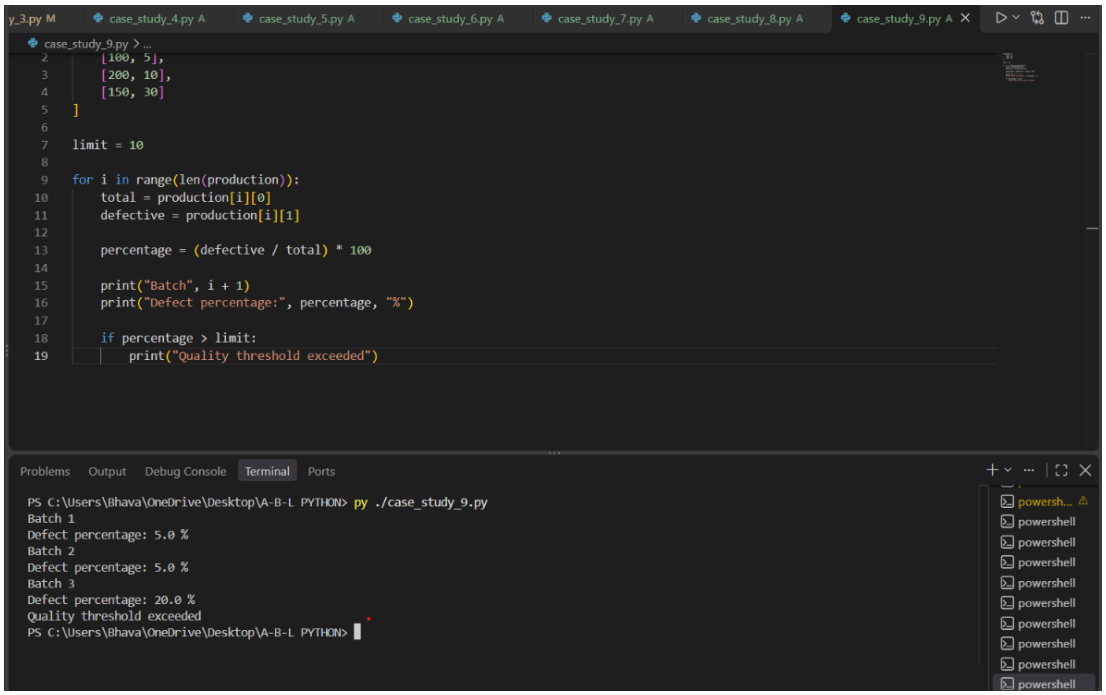
7. Pseudocode

```
BEGIN
  production = rows of [total units, defective units]
  limit = 10

  FOR each batch at index i
    total = production[i][0]
    defective = production[i][1]
    percentage = (defective / total) * 100
    display batch number and percentage
    IF percentage > limit
      display quality threshold warning
    END IF
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows defect percentages of 5% for Batch 1, 5% for Batch 2, and 20% for Batch 3. Since only Batch 3 exceeds the 10% limit, it receives the “Quality threshold exceeded” warning.



9. Conclusion

The program converts batch defect counts into percentages, making results comparable across different production volumes. In the sample, Batches 1 and 2 are below the 10% limit, while Batch 3 has a 20% defect rate and is flagged. This provides a basic way to direct quality review toward batches with higher defect rates.

MULTI-BRANCH EXPENSE MATRIX ANALYSIS

Python Programming Case Study

1. Case Study

This case study analyzes expense data for multiple business branches. The program stores each branch’s expense categories in a nested list, then calculates and displays the total and average expense for each branch.

2. Problem Statement

Comparing expenses across branches requires adding several category values for each location. Manual calculations can be slow and prone to error. A program is needed to summarize each branch’s spending consistently.

3. Objective

To use a two-dimensional list and iteration to calculate the total expense and average expense for each branch, then print the results.

4. Assumptions

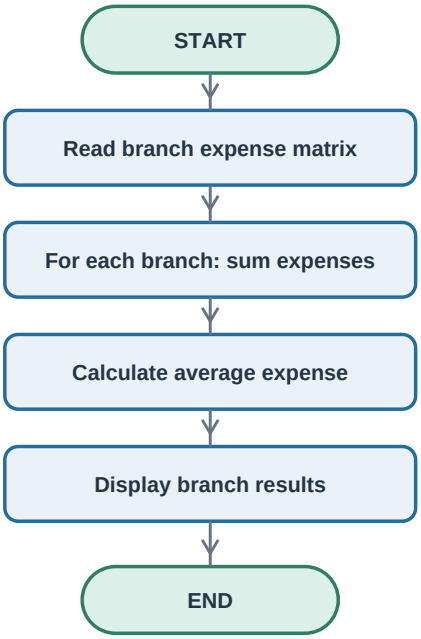
• Each row in the expense matrix represents one branch.
• Each value in a row is an expense category for that branch.
• Expense values are numeric and use the same currency and time period.
• Each branch has at least one expense value, so its average can be calculated.
• The average is the arithmetic mean of the listed category amounts.

5. Algorithm

1. Store each branch’s expense values as a row in a list.
2. Loop through the rows using the branch index.
3. Add the row values with **sum()** to find the total.
4. Divide the total by the number of values in the row to find the average.
5. Display the branch number, total expense, and average expense.

6. Flowchart

The program processes each row of the expense matrix and reports the calculated total and average.



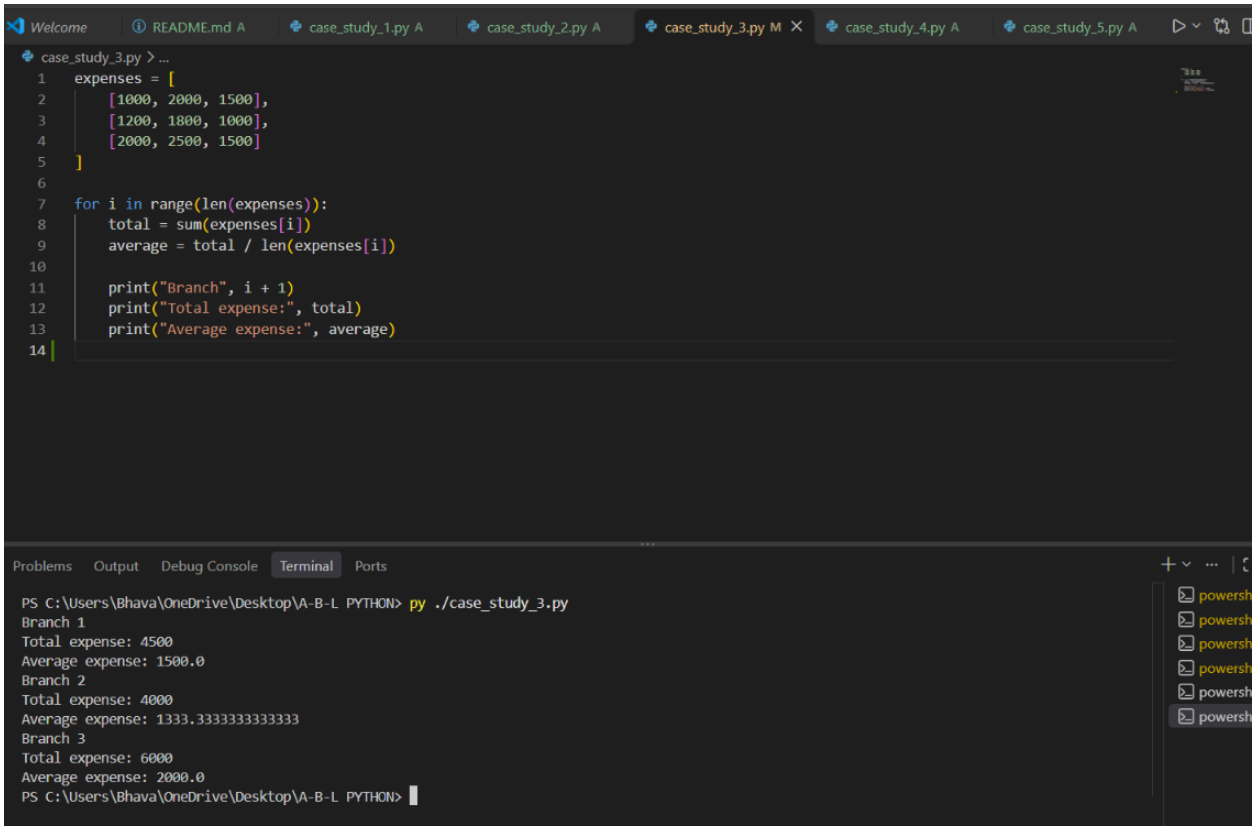
7. Pseudocode

```
BEGIN
  expenses = rows of expense values, one row per branch

  FOR each branch row at index i
    total = sum(expenses[i])
    average = total / length(expenses[i])
    display branch number i + 1
    display total and average
  END FOR
END
```

8. Program and Output Screenshot

The screenshot shows the source code and output. Branch 1 has a total expense of 4,500 and average of 1,500. Branch 2 has a total of 4,000 and average of about 1,333.33. Branch 3 has a total of 6,000 and average of 2,000.



9. Conclusion

The program summarizes the expense categories for each branch and calculates an average, making branch-level spending easier to review. In the sample data, Branch 3 has the highest total and average expenses, while Branch 2 has the lowest total. The same method can be applied to larger expense matrices.

NESTED LIST EXAMINATION SEATING VALIDATION

Python Programming Case Study

1. Case Study

This case study validates examination seating data stored as a nested list. Each inner list represents a row of seats, where a roll number identifies an assigned student and None represents an empty seat. The program reports empty seats, detects repeated roll numbers, and records unique roll numbers during the scan.

2. Problem Statement

Exam seating plans must not assign the same student to more than one seat, and unfilled seats should be identified. Manually checking a seating grid can miss duplicates or empty positions. A program can scan every seat systematically and report these conditions.

3. Objective

To traverse a nested seating list, report empty seats, detect duplicate student roll numbers, and confirm when validation has completed.

4. Assumptions

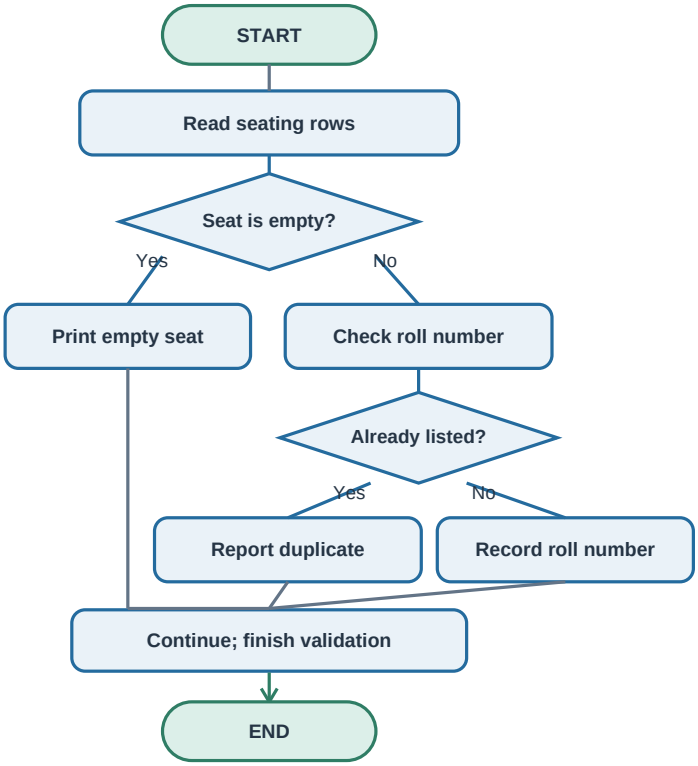
- The outer list contains seating rows and each inner list contains seats in that row.
- A seat contains either None or a student roll number.
- Roll numbers can be compared for equality and are expected to be unique across occupied seats.
- Empty seats are reported but are not added to the list of validated students.
- The program reports issues; it does not automatically change the seating plan.

5. Algorithm

1. Store the seating arrangement as a nested list.
2. Create an empty list to track roll numbers already encountered.
3. Visit each row and then each seat in that row.
4. If a seat is None, print “Empty seat.”
5. Otherwise, if the roll number has already been recorded, print a duplicate warning.
6. If it is a new roll number, add it to the tracking list.
7. After all seats are checked, print “Seating validation completed.”

6. Flowchart

For every seat, the program checks whether it is empty, then checks for a repeated roll number before continuing.



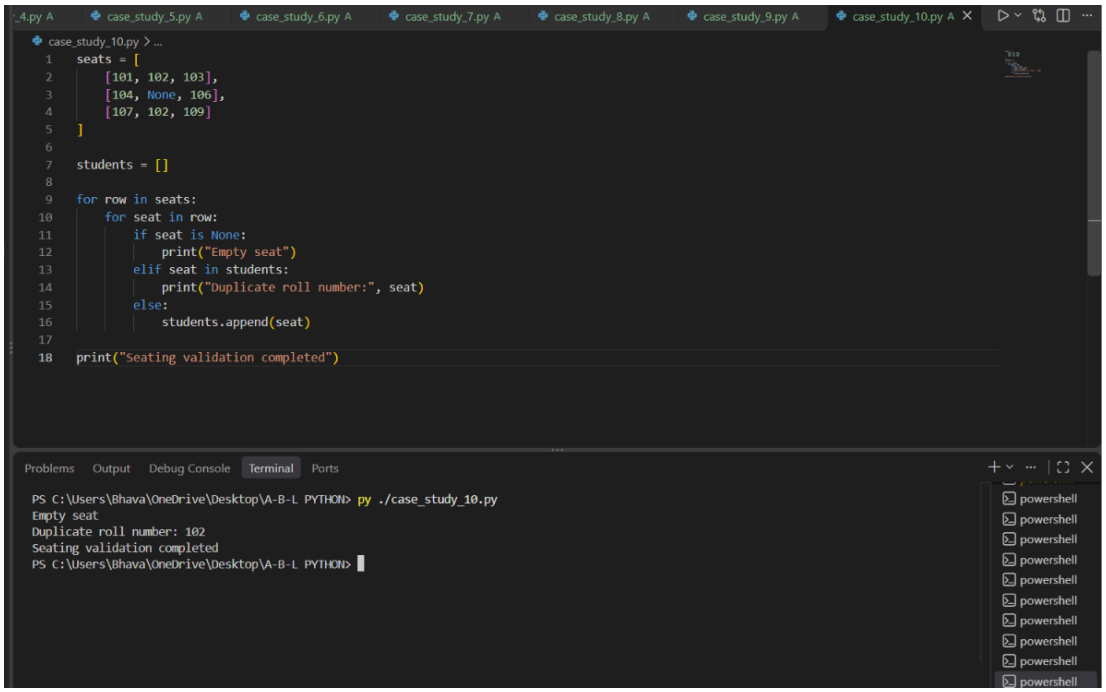
7. Pseudocode

```
BEGIN
  seats = nested list of seat values
  students = empty list

  FOR each row in seats
    FOR each seat in row
      IF seat is None
        display empty seat
      ELSE IF seat is in students
        display duplicate roll number
      ELSE
        add seat to students
      END IF
    END FOR
  END FOR
  display validation completed
END
```

8. Program and Output Screenshot

The screenshot shows a 3 by 3 seating list. The program reports one empty seat and identifies roll number 102 as a duplicate, then confirms that seating validation is completed.



9. Conclusion

The nested-list scan checks every seat and identifies both missing assignments and repeated roll numbers. In the sample, one seat is empty and roll number 102 appears more than once. This validation can help staff review seating plans before an examination, while leaving corrections to the responsible administrator.

SMART CANTEEN ORDER AND STOCK MANAGEMENT

Python Programming Case Study

1. Case Study

This case study models a small canteen that processes a list of food orders. For each item, the program checks whether the requested quantity is available, subtracts fulfilled quantities from stock, calculates the bill using the item price, and accumulates total sales. It then displays remaining stock.

2. Problem Statement

A canteen must bill orders accurately while avoiding sales beyond available inventory. Manually checking stock, calculating each order cost, and updating inventory can lead to mistakes. A simple program can automate these steps and report items that cannot be fulfilled.

3. Objective

To process item and quantity pairs against a stock dictionary, calculate bills for available orders, update stock, report insufficient stock, and display total sales and remaining inventory.

4. Assumptions

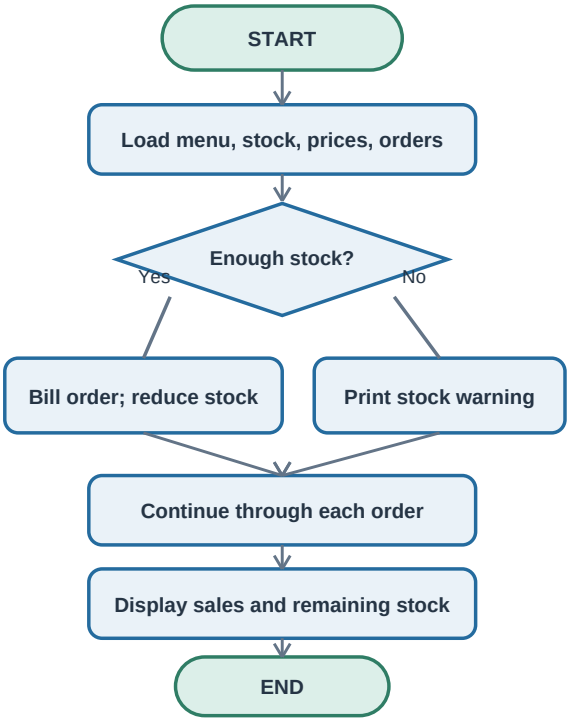
• Each order contains a menu item name and a positive quantity.
• Item names in orders exist in both the price and stock dictionaries.
• Stock and prices are numeric; prices are per unit and use the same currency.
• An order is fulfilled only when the full requested quantity is available.
• If stock is insufficient, the program prints a message and leaves that item’s stock and sales unchanged.

5. Algorithm

1. Define the stock available and price for each menu item.
2. Initialize total sales to zero and store the orders.
3. For each order, compare its quantity with the available stock.
4. If enough stock is available, subtract the quantity from stock, calculate price × quantity, add the bill to sales, and print the bill.
5. Otherwise, print an insufficient-stock message.
6. After processing all orders, print total sales and the remaining stock.

6. Flowchart

For each order, the program branches based on stock availability, then continues until it can display the final sales and inventory.



7. Pseudocode

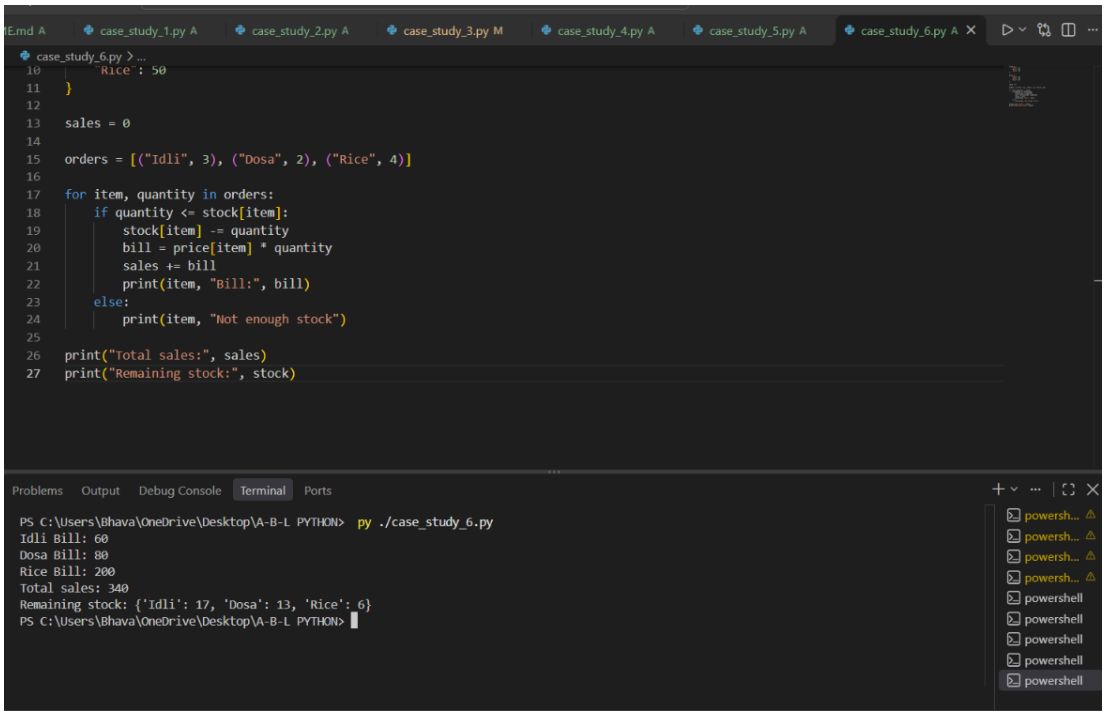
```
BEGIN
  define stock and price for each item
  sales = 0
  orders = list of (item, quantity) pairs

  FOR each (item, quantity) in orders
    IF quantity <= stock[item]
      stock[item] = stock[item] - quantity
      bill = price[item] * quantity
      sales = sales + bill
      display item and bill
    ELSE
      display insufficient stock message
    END IF
  END FOR

  display sales and remaining stock
END
```

8. Program and Output Screenshot

The screenshot shows the program processing Idli, Dosa, and Rice orders. It prints bills of 60, 80, and 200, for total sales of 340. Remaining stock is Idli 17, Dosa 13, and Rice 6.



9. Conclusion

The program combines order billing with inventory tracking. For the sample orders, it correctly calculates total sales of 340 and updates each item's stock after fulfillment. The insufficient-stock branch also provides a clear warning when an order cannot be completed, helping the canteen avoid deducting stock for an unfulfilled order.

STUDENT GROUP FORMATION WITH SCORE BALANCING

Python Programming Case Study

1. Case Study

This case study divides an ordered list of student scores into two groups by alternating positions. Group 1 receives the scores at even indexes and Group 2 receives the scores at odd indexes. The program prints both groups and their total scores so their score balance can be reviewed.

2. Problem Statement

When students are assigned to groups, a large difference between group score totals may indicate an uneven distribution. A simple method is needed to split the scores and compare the combined scores of the resulting groups.

3. Objective

To use Python list slicing to create two groups from an ordered score list and calculate the total score of each group.

4. Assumptions

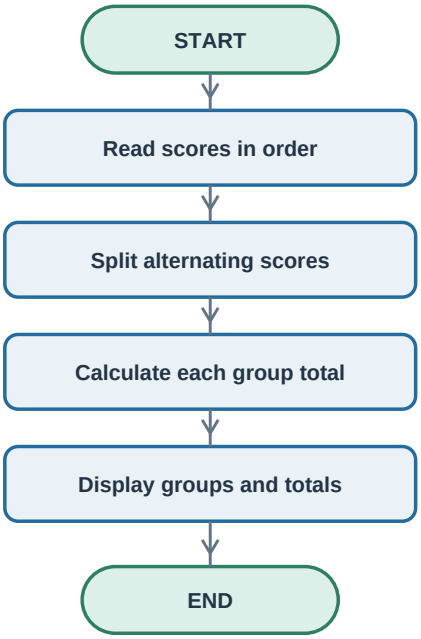
• Each value represents one student’s score and is numeric.
• Scores are taken in their existing order; the program does not sort them first.
• Group 1 receives indexes 0, 2, 4, and so on; Group 2 receives indexes 1, 3, 5, and so on.
• The number of students may be odd, so group sizes can differ by one.
• Alternating assignment is a simple distribution method and does not guarantee the closest possible group totals for every input.

5. Algorithm

1. Store the student scores in a list.
2. Use the slice scores[::2] to create Group 1.
3. Use the slice scores[1::2] to create Group 2.
4. Display both groups.
5. Calculate and display each group’s total using sum().

6. Flowchart

The program splits scores by alternating list positions, sums each group, and displays both results.



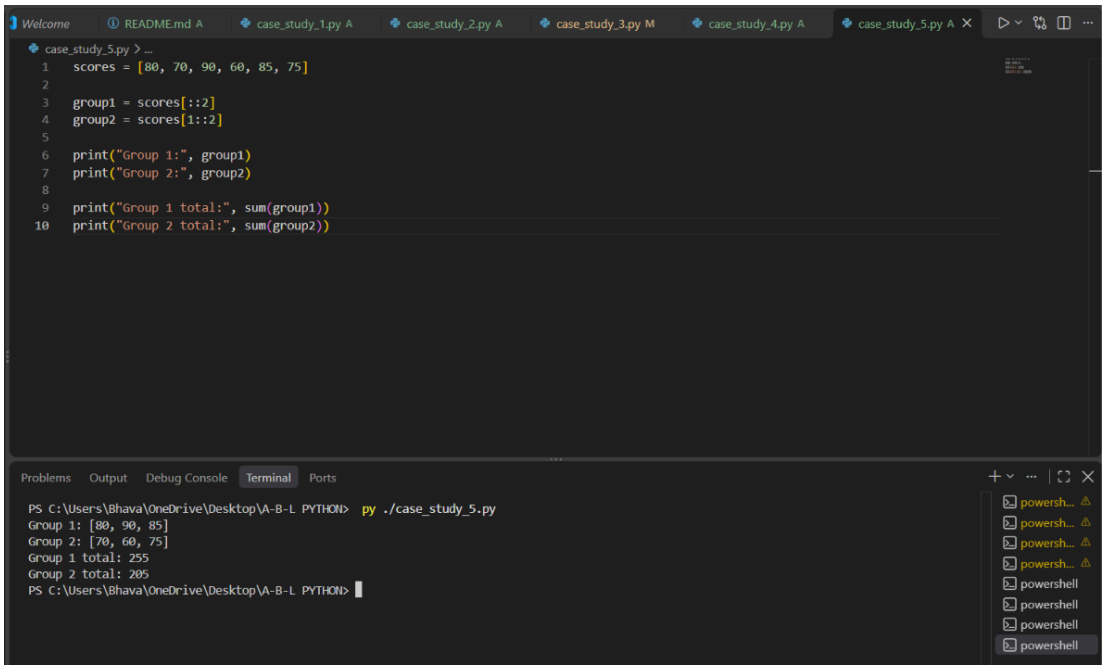
7. Pseudocode

```
BEGIN
  scores = ordered list of student scores
  group1 = scores from index 0, taking every 2nd value
  group2 = scores from index 1, taking every 2nd value

  display group1
  display group2
  display sum(group1)
  display sum(group2)
END
```

8. Program and Output Screenshot

The screenshot shows scores [80, 70, 90, 60, 85, 75]. Alternating slicing produces Group 1 [80, 90, 85] with a total of 255, and Group 2 [70, 60, 75] with a total of 205.



9. Conclusion

The program uses alternating list slices to form two groups and compares their score totals. In the sample, Group 1 totals 255 and Group 2 totals 205, a difference of 50 points. This approach is straightforward, but its result depends on the order of the input scores; sorting or optimization would be needed to seek a closer balance in other datasets.

TUPLE-BASED TRANSACTION AUDIT

Python Programming Case Study

1. Case Study

This case study demonstrates a basic transaction audit using Python tuples. Each transaction contains an ID, an amount, and a transaction type. The program checks the records for repeated IDs and flags transactions with negative amounts, then reports that the audit is complete.

2. Problem Statement

Transaction records may contain duplicate identifiers or invalid negative amounts. Manually reviewing a list of records can be error-prone. A simple program can systematically flag these issues so they can be investigated.

3. Objective

To iterate through a list of transaction tuples, detect duplicate transaction IDs, flag negative amounts, and print an audit completion message.

4. Assumptions

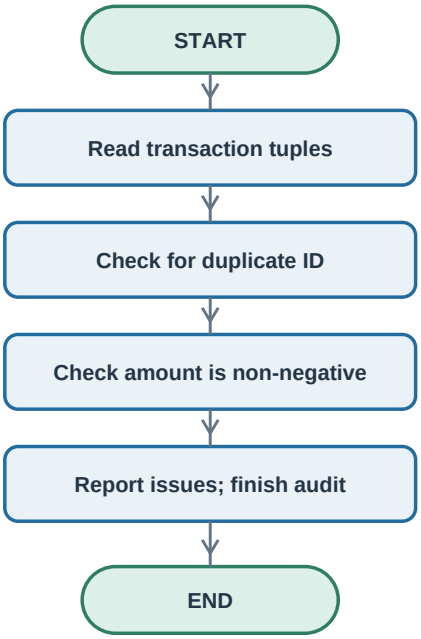
• Each transaction is a tuple in the order (ID, amount, type).
• Transaction IDs can be compared for equality.
• A repeated ID is reported as a duplicate; the program still continues checking the record.
• A negative amount is considered invalid.
• This example checks duplicates and negative amounts only; it does not validate transaction types or reconcile balances.

5. Algorithm

1. Store the transactions as tuples in a list.
2. Create an empty list to hold transaction IDs already seen.
3. For each transaction, check whether its ID is already in the ID list.
4. If it is already present, print a duplicate-ID warning; otherwise, add the ID to the list.
5. Check whether the amount is less than zero and print an invalid-amount warning when it is.
6. After all records are checked, print “Audit completed.”

6. Flowchart

Each record is checked for a duplicate ID and a negative amount before the audit finishes.



7. Pseudocode

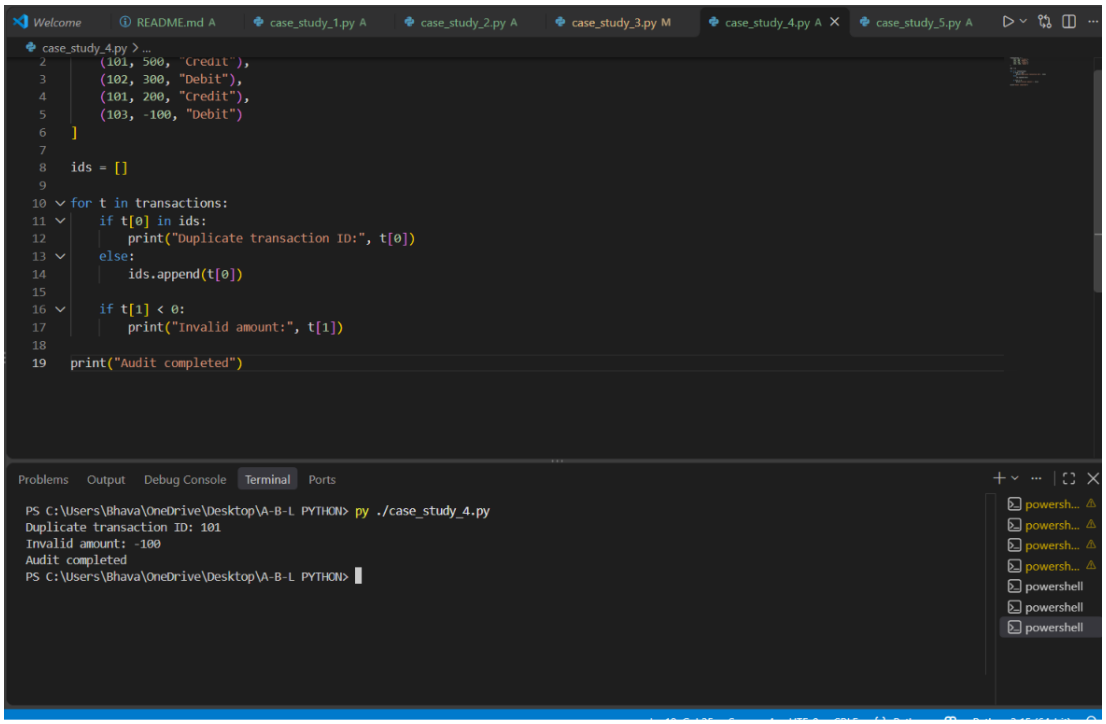
```
BEGIN
  transactions = list of (ID, amount, type) tuples
  ids = empty list

  FOR each transaction t
    IF t.ID is in ids
      display duplicate transaction ID
    ELSE
      append t.ID to ids
    END IF

    IF t.amount is less than 0
      display invalid amount
    END IF
  END FOR
  display audit completed
END
```

8. Program and Output Screenshot

The screenshot shows the source code and terminal output. The audit reports transaction ID 101 as a duplicate and flags an amount of -100 as invalid, then prints “Audit completed.”



9. Conclusion

The tuple-based audit checks each transaction for two common data issues: duplicate identifiers and negative amounts. In the displayed example, it correctly identifies both issues and reaches completion. Additional validation, such as checking transaction types or ensuring IDs are unique across a larger data source, could extend the audit.